

Summary of Notation

1. Capital letters: random variables
2. Lowercase letters: values of random variables & for scalar functions.
3. **Quantities** that are required to be real-valued vectors are written in **bold** and in **lowercase** (even if random variables). Matrices are bold capitals.
4. Abbreviations: - for disadvantages, + for advantages, sol for solution, nw for network

Symbols

- $\doteq$: equality relationship that is true by definition
- $\approx$: approximately equal
- $\propto$: proportional to
- $Pr\{X = x\}$: probability that a random variable X takes on the value x
- $X \sim p$: random variable X selected from distribution $p(x)$
- $E[X]$: expectation of a random variable X , i.e.,
 $E[X] \doteq \sum_x p(x)x$
- $\arg \max_a f(a)$: a value of a at which $f(a)$ takes its maximal value
- $\ln x$: natural logarithm of x
- e^x : the base of the natural logarithm, $e \approx 2.71828$, carried to power x ; $e^{\ln x} = x$
- $\mathbb{R}$: set of real numbers
- $f : X \rightarrow Y$: function f from elements of set X to elements of set Y
- $(a, b]$: the real interval between a and b including b but not including a

Parameters

- ϵ : probability of taking a random action in an ϵ -greedy policy
- α, β : step-size parameters
- γ : discount-rate parameter
- λ : decay-rate parameter for eligibility traces
- $1_{\text{predicate}}$: indicator function. = 1 if the predicate is true, else 0.

1 Introduction

Approximations for value f, policy, models

Even if the agent has a complete and accurate environment model, the agent is typically unable to perform enough computation (memory) per time step to fully use it.

2 Multi-Armed Bandit Problem

- k : number of actions (arms)
- t : discrete time step or play number
- $q_*(a)$: true value (expected reward) of action a
- $Q_t(a)$: estimate at time t of $q_*(a)$
- $N_t(a)$: number of times action a has been selected up to time t
- $H_t(a)$: learned preference for selecting action a at time t
- $\pi_t(a)$: probability of selecting action a at time t
- $\bar{R}_t$: estimate at time t of the expected reward given π_t

3 Markov Decision Process

TODO: 寻找最优 π 的方法。

3.1 MDP concepts

The reinforcement learning agent and its environment interact over a sequence of discrete time steps. Everything inside the agent is completely known and controllable by the agent; everything outside is incompletely controllable but may or may not be completely known. The agent's objective is to maximize the amount of reward it receives over time.

white-box environment -> the optimal policy can be solved by the dynamic programming: PI, VI

black-box -> model-free RL methods

- t : discrete time step.
- s, s' : current state, next state, basis for making the choices
- a : an action made by the agent
- r : a reward, basis for evaluating the choices
- S : set of all nonterminal states
- S^+ : set of all states, including the **terminal state**
- $A(s)$: set of all actions available in state s
- R : set of all possible rewards, a finite subset of $\mathbb{R}$
- $\rho \subseteq \mathbb{R}$: subset of $\mathbb{R}$
- $|S|$: number of elements in set S
- $T, T(t)$ final time step of an episode; the episode including time step t .
- $A(t)$: Action at t
- $S(t)$: state at t
- $R(t)$: reward at t
- π : policy (stochastic decision-making rule), is a mapping from states to actions
- $\pi(s)$: action taken in state s under deterministic policy π
- $\pi(a|s)$: probability of taking action a in state s under stochastic policy π

Stochastic process: 1/more events, stochastic(dynamic) system/phenomenon evolve with t .

$$P[S_{t+1}|S_1, \dots, S_t]$$

Markov process

- $\langle S \text{ state}, P \rangle$
- a stochastic process
- with Markov property: The current state is the future sufficient statistics (充分统计量). (*future ⊥ past*) | *present*

$$P[S_{t+1}|S_t] = P[S_{t+1}|S_1, \dots, S_t]$$

Markov reward process(MRP)

- $\langle S, P, r \text{ reward}, \gamma \text{ discount} \rangle$
- env: 不受agent控制, 产生 s, r
- episodes: agent-environment interaction breaks naturally into subsequences, each episode ends in a special state called the terminal state.
- **discounted episode's return from s_t**

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k} = R_t + \gamma G_{t+1}$$

$\gamma \in [0, 1]$, which helps to converge.

Comparing the G here and the one in book ($G'_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + R_T = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$), G here makes sure the total order of episodic return for all different return sequences.

$$(R_1, R_2, R_3, R_4) = 0, 1, 0, 1 \quad \textcircled{1} \quad \gamma = 0.1$$

$$(R'_1, R'_2, R'_3, R'_4) = 1, 1, 0, 1 \quad \textcircled{2}$$

✓ 殊 $G_{1\textcircled{1}} = 0 + 0.1 \times 1 + 0.01 \times 0 + 0.001 \times 1 = 0.1 + 0.001 = 0.101$
 $G_{1\textcircled{2}} = 1 + \dots = 1.101$ } 可以区分

书 $G'_{10} = 1 + 0.1 \times 0 + 0.001 \times 1 \parallel$
 $G'_{12} = 1 + \dots$ } 无法区分

if 要保证2个任意回报序列可以比较(有全序)

$$\& \text{满足 } [a_1, a_2, \dots] < [b_1, b_2, \dots] \Leftrightarrow [r, a_1, a_2, \dots] < [r, b_1, b_2, \dots]$$

我取某时 r 小于另一个

无论在前面加上任何相同 r , 比较结果不变.

$\Rightarrow$ 唯一确定 $\rightarrow$ 计算方式就是 discount summation
 递归地保持相同顺序

proof: $a_0 + r_1 a_1 > b_0 + r_1 b_1$

$$r + r_1 a_0 + r_2 a_1 > r + r_1 b_0 + r_2 b_1$$

$$(a_0 - b_0) r_1 + (a_1 - b_1) r_2 > 0$$

$$\text{下项 } (a_1 - b_1) r_2 + (a_2 - b_2) r_3 > 0$$

3. 设等比数列形式

我们说:

$$\gamma_2 = r \gamma_1$$

$$\gamma_3 = r \gamma_2 = r^2 \gamma_1$$

$$\gamma_4 = r \gamma_3 = r^3 \gamma_1$$

这样我们就得到:

$$\gamma_t = r^{t-1} \gamma_1$$

如果我们设 $r = \gamma_1$, 那么就变成:

$$\gamma_t = \gamma_1^t$$

这正是一个标准的等比数列!

Markov decision process (MDP)

- $\langle S, A, P, r, \gamma \rangle$
- state transfer function $p(s'|s, a) = \Pr\{S_t = s' | S_{t-1} = s, A_{t-1} = a\}$
- reward function $r(s, a)$



与书定义不一样.

- dynamics: MDP stochastic transition, to next state $s \sim P(\cdot | s_0, a_0)$. dot $(\cdot)$ represents any possible next state

$$s_0 \xrightarrow{a_0, r(s_0, a_0)} s_1 \xrightarrow{a_1, r(s_1, a_1)} s_2 \xrightarrow{a_2, r(s_2, a_2)} s_3 \dots s_T$$

Termination

$$\text{Total return } r(s_0, a_0) + \gamma r(s_1, a_1) + \gamma^2 r(s_2, a_2) + \dots$$

eg. 只与 state 相关 reward: maze, go

$$\text{Total ret } r(s_0) + \gamma r(s_1) + \gamma^2 r(s_2) + \dots$$

❑ In most cases, rewards are only related to state:

- ❑ In maze, rewards are only related to location
- ❑ In Go, rewards are only based on the size of the final territory

❑ Reward function: $r(s): \mathcal{S} \rightarrow \mathbb{R}$

❑ MDP:

$$s_0 \xrightarrow{a_0, r(s_0)} s_1 \xrightarrow{a_1, r(s_1)} s_2 \xrightarrow{a_2, r(s_2)} s_3 \dots$$

❑ cumulative reward:

$$r(s_0) + \gamma r(s_1) + \gamma^2 r(s_2) + \dots$$

Policy $\pi(a|s) = P(a|s)$

- prob of taking action a in state s
- Markov property: policy needs to be related to current s; does not consider historical s.
- For 2 kinds of policies:

$$\sum_a \pi(a|s) = 1$$

- deterministic policy: for a given s, the agent always chooses the same a, without any randomness or probabilities involved.

$$\pi(a|s) = \pi(s) = \mu(s) = \begin{cases} 1 & \text{if } a = \mu(s) \\ 0 & \text{otherwise} \end{cases}$$

- stochastic policy: a is sampled

$$\pi(a|s) \in [0, 1]$$

MDP to MRP: marginalization

- MRP是MDP的简化版本, 去掉了动作部分, 只关注状态转移、奖励函数和折扣因子 (γ)。
- 这有助于分析和处理那些不需要明确决策行为的系统, 或者是MDP的价值评估阶段。
- 在每个状态下, 按照某个策略选择动作后, 状态的转移概率:

$$P(s'|s) = \sum_a \pi(a|s) P(s'|s, a)$$

- reward at s

$$r(s) = \sum_a \pi(a|s) r(s, a)$$

So, MRP $\langle S, P', r', \gamma \rangle$

Data distribution

- Given the same MDP, the state-action(s-a) distribution sampled by different policy is different.
- Observing the path of MDP..
- state dist $d^\pi(s) = \sum_{t=0}^{\infty} P(s|\pi)$
- state-action dist $d^\pi(s, a) = \sum_{t=0}^{\infty} P(s, a|\pi)$
- Relationship

$$d^\pi(s) = \sum_a d^\pi(s, a)$$

$$d^\pi(s, a) = d^\pi(s) \pi(a|s) \text{ (Bayes)}$$

State-action occupancy measure

- describes how often a policy visits certain state-action pairs in a Markov Decision Process (MDP).
- provides a way to express the long-term distribution of states and actions under a given policy.
- Discounted state-action distribution:

$$\hat{\rho}^\pi(s, a) = \mathbb{E}_n \left[\sum_{t=0}^{\infty} \gamma^t \mathbb{I}(s, a) \middle| S_t = s, A_t = a \right] \quad \forall s \in S, a \in \mathcal{A}$$

$$= \sum_{t=0}^{\infty} \gamma^t P(s, a | \pi)$$

- Discounted state distribution:

$$\hat{v}^\pi(s) = \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(S_t = s | \pi)$$

$$= \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(s | \pi) \sum_a \pi(a | s)$$

$$= \sum_{t=0}^{\infty} \sum_a \gamma^t \mathbb{P}(s | \pi) \sum_a \pi(a | s)$$

$$= \sum_a \sum_{t=0}^{\infty} \mathbb{P}(s, a | \pi)$$

$$= \sum_{t=0}^{\infty} \hat{\rho}^\pi(s, a)$$

- Normalization (When $(\gamma < 1)$): two distribution above -> real dist

$$\sum_s \hat{v}^\pi(s) = \sum_s \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(s | \pi)$$

$$= \sum_{t=0}^{\infty} \sum_s \gamma^t \mathbb{P}(s | \pi)$$

$$= \sum_{t=0}^{\infty} \gamma^t$$

$$= \frac{1}{1 - \gamma}$$

- Normalized Discounted (Action-) State Distribution:

$$v^\pi(s) = (1 - \gamma) \hat{v}^\pi(s)$$

$$\rho^\pi(s, a) = (1 - \gamma) \hat{\rho}^\pi(s, a)$$

- relationship

$$\rho^\pi(s, a) = v^\pi(s) \pi(a | s)$$

$$v^\pi(s) = \sum_a \rho^\pi(s, a)$$

- Theorem 1: $\rho^{\pi_1} = \rho^{\pi_2}$ iff $\pi_1 = \pi_2$

- Theorem 2: $\pi_\rho(s|a) = \frac{\rho(s,a)}{\sum_a \rho(s,a)}$

◦ Given occupancy measure ρ , The only policy that can generate this occupancy metric is this formula.

- Policy on occupancy measure

◦ policy cumulative reward

$$J(\pi) = E_\pi \left[\sum_{t=0}^{\infty} \gamma^t r(S_t, A_t) \right]$$

$$\begin{aligned}
&= \sum_{t=0}^{\infty} \gamma^t E_{\pi}[r(S_t, A_t)] \\
&= \sum_{t=0}^{\infty} \gamma^t \sum_a \sum_s P(s, a | \pi) \mathbf{r}(s, a) \\
&= \sum_a \sum_s \left[\sum_{t=0}^{\infty} \gamma^t P(s, a | \pi) \right] \mathbf{r}(s, a) \\
&= \sum_a \sum_s \hat{\rho}^{\pi}(s, a) \mathbf{r}(s, a) \\
&= E_{(s,a) \sim \hat{\rho}^{\pi}}[r(s, a)]
\end{aligned}$$

- Policy learning goal

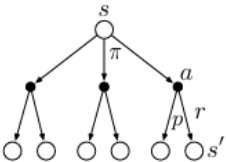
$$\max_{\pi} J(\pi) = E_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r(S_t, A_t) \right] \simeq E_{(s,a) \sim \hat{\rho}^{\pi}}[r(s, a)]$$

Bellman expectation equations

- $V^{\pi}(s)$: Expected reward by following policy π from state s
- $Q^{\pi}(s, a)$: Expected reward while following policy π from state s and take action a
 - q : quality

$$\begin{aligned}
V^{\pi}(s) &= E_{\pi}[G_t | s] \text{ def} \\
&= E_{\pi}[R_t + \gamma G_{t+1} | s] \\
&= E_{\pi}[R_t | s] + E_{\pi}[\gamma G_{t+1} | s] \\
&= E_{\pi}[R_t | s] + \gamma E_{\pi}[G_{t+1} | s] \\
&= E_{\pi}[R_t | s] + \gamma V^{\pi}(s_{t+1}) \\
&= E_{\pi}[R_t | s] + E_{\pi}[\gamma V^{\pi}(s_{t+1}) | s] \\
&= \sum_a \pi(a | s) r(s, a) + \sum_a \pi(a | s) \gamma \sum_{s_{t+1}} P(s_{t+1} | s, a) V^{\pi}(s_{t+1}) \\
&= \sum_a \pi(a | s) (r(s, a) + \gamma \sum_{s_{t+1}} P(s_{t+1} | s, a) V^{\pi}(s_{t+1})) \\
&= \sum_a \pi(a | s) Q(s, a) \\
\\
Q^{\pi}(s, a) &= E_{\pi}[G_t | s, a] \\
&= E_{\pi}[R_t + \gamma G_{t+1} | s, a] \\
&= E_{\pi}[R_t + \gamma Q(s_{t+1}, A_{t+1}) | s, a] \\
&= E_{\pi}[R_t | s, a] + E_{\pi}[\gamma Q(s_{t+1}, A_{t+1}) | s, a] \\
&= r(s, a) + \gamma \sum_{s_{t+1}} P(s_{t+1} | s, a) \sum_{a_{t+1}} \pi(a_{t+1} | s_{t+1}) Q(s_{t+1}, a_{t+1}) \\
&= r(s, a) + \gamma \sum_{s_{t+1}} P(s_{t+1} | s, a) V^{\pi}(s_{t+1}) \\
&= \mathbf{r}(s, a) + \gamma E_{s' \sim P(\cdot | s, a)}[V^{\pi}(s')]
\end{aligned}$$

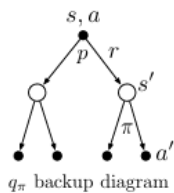
- **Explanations** The value function V_{π} 's Bellman equation.



Backup diagram for v_{π}

Starting from state s , the root node at the top, the agent could take any of some set of actions (e.g.3) based on its policy π . From each of these, the environment could respond with one of several next states, s_0 (e.g.2), along with a reward, r , depending on its dynamics given by the function p .

- The action-value function Q_{π} 's Bellman equation.



If we were to take action a in state s , what is the expected return if we then follow π afterward?

3.2 Policy evaluation

Policy improvement theorem

Assume deterministic policy π

Policy π' is the improvement of π if: for any s , $Q^\pi(s, \pi') \geq V^\pi(s)$

Then π and π' satisfy: for any s , $V^{\pi'}(s) \geq V^\pi(s)$

PROOF 28

3.3 Find optimal policy

3.3.1 Policy iteration

Given a MDP with limited action space and state space: $|S| < \infty, |A| < \infty$

- PI based on s value V

1. Randomly initialize policy π
2. Repeat until convergence {
 - a) Calculate $V = V^\pi$ (Updating is time consuming)
 - b) For each state $s \in S$, update:

$$\pi(s) = \arg \max_a r(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s')$$

}

- PI based on action value Q

1. Randomly initialize policy π
2. Repeat until convergence {
 - a) Calculate $Q = Q^\pi$ (Updating is time consuming)
 - b) For each state $s \in S$, update:

$$\pi(s) = \arg \max_a Q(s, a)$$

}

- PROOF 35,36 at k

3.3.2 Value iteration + e.g.

Speed up V , same for Q :

Policy evaluation is time consuming. But in the previous example, when the iteration of value function V is not convergent, the derived policy is already optimal. Don't wait until convergence! If only one round of value update is carried out in the policy evaluation, and then the policy is upgraded directly according to the updated value.

1. For each state, initialize $V(s)=0$
2. Repeat until convergence {
 - a) For each state $s \in S$, update:

$$V(s) = \arg \max_a r(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s')$$

}

3. Return a deterministic policy

$$\pi(s) = \arg \max_a r(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s')$$

Synchronous value iteration store two copies of value function (Update new by using old)

Asynchronous value iteration only store one copy of value function (Update new by using both old and new)

3.3.3 Optimal policy π^*

In MDP with limited state and action space, there is a policy:

$$V^{\pi^*}(s) \geq V^{\pi}(s) \text{ for any } s \in S$$

Optimal state value f

$$\begin{aligned} V^*(s) &= \max_{\pi} V^{\pi}(s) \\ &= \max_{\pi} \sum_a \pi(a|s) Q^{\pi}(s, a) \\ &= \max_a Q^*(s, a) \\ &= \max_a r(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') \end{aligned}$$

Optimal action value f

$$\begin{aligned} Q^*(s, a) &= \max_{\pi} Q^{\pi}(s, a) \\ &= r(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') \\ &= r(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_a Q^*(s, a) \end{aligned}$$

When value function is optimal, its policy is optimal:

$$\pi^*(s) = \arg \max_a r(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s')$$

relationship

Proof: Value Iteration

3.4 Policy v.s. Value Iteration

Value iteration: greedy update method, equivalent to a round of value update in policy evaluation, and then the policy is upgraded directly according to the updated value

In policy iteration, the cost of updating value function by using Bellman equation is very large!

For MDP with small space, policy iteration usually converges quickly

For MDP with large space, value iteration is more practical, more efficient

If no state transition loop, best to use value iteration

3.5 Model-based method

MDP $\langle S, A, P, r, \gamma \rangle$.

In real applications, P and r are unknown.

Need to learn the state transition probability P and reward r

$$P(s'|s, a) = \frac{\# \text{ take a under } s \text{ and transfer to } s'}{\# \text{ take a under } s}$$

$$r(s, a) = \text{average } r(s, a)^{(i)}$$

Algorithm

1. Randomly initialize policy π
2. Repeat until convergence {
 - a) Execute π in MDP, collect experienced data
 - b) Use the collected experience in MDP to update the estimation of P and r
 - c) Value iteration by using the estimation of P and r to get new estimation of value function V
 - d) Update policy π as greedy policy according to V

$\therefore$ action space is too large. $\rightarrow$ model-free

4 Model-free method

If a problem cannot be modeled as MDP, it is not a reinforcement learning problem.

Can be modeled does not mean that we know all the parameters.

Here comes the model-free method:

Learn value function and policy directly from experiences.

Key steps:

- (1) Estimating value function (Policy evaluation)
- (2) Optimizing policy (Policy improvement)/ control

2 kinds of policy learning

on-policy learning: Sampling policy and learning policy is the same!

off-policy: Sampling policy and learning policy is different!

- target policy $\pi(a|s)$: evaluate value function $V^\pi(s)$ or $Q^\pi(s, a)$
- Behavior policy $\mu(a|s)$: collect data $\{s_t, a_t, r_t, s_{t+1}, \dots, s_{T-1}, a_{T-1}, r_{T-1}, s_T, a_T, r_T\} \sim \mu$
- why:
 - Balance exploration and exploitation
 - Learning policy by observing human beings or other agents
 - Experience from reusing old policies
 - Learn the optimal policy when following the policy used in exploration
 - Learn multiple policies when following one policy in exploration
 - An example of MSR research in Cambridge

4.1 Monte Carlo value estimation

MC: repeated random sampling to obtain numerical results.

can only be applied to MDP with finite length (all episode has a terminate state).

learns from the complete episode: no bootstrapping

work in a fragmented (terminated) environment.

must wait for the end of the episode until the cumulative reward is known

Steps:

1. Sampling a lot of episodes using policy π
2. For the state s of each time step t in each episode: Incremental counter $N(s) \leftarrow N(s) + 1$

Incremental total cumulative reward $M(s) \leftarrow M(s) + G_t$

Value is estimated as the average of the cumulative rewards $V(s) = M(s)/N(s)$

3. According to the law of large numbers: $V(s) \rightarrow V^\pi(s)$ as $N(s) \rightarrow \infty$

Update $V(s)$ immediately after sampling an episode

4. For each state s_t and its cumulative reward g_t

$$N(s_t) \leftarrow N(s_t) + 1$$
$$V(s_t) \leftarrow V(s_t) + \frac{1}{N(s_t)}(g_t - V(s_t))$$

For nonn-stationary problems (that is, the environment will change over time), we can track a current average (that is, we don't consider episodes that are too long ago). Take α as a constant.

$$V(s_t) \leftarrow V(s_t) + \alpha(g_t - V(s_t))$$

Analysis:

The cummulative reward g_t is the unbiased estimation of $V(s_t)$

Good convergence property (This is still true when using functional approximation)

Insensitive to initial values

Easy to understand and use

4.2 Temporal difference

By bootstrapping, TD learns from incomplete fragment.

updates the current prediction value to make it close to the estimated cumulative reward (untrue value).

works in a continuous (non-terminating) environment.

can do real-time learning at each step.

Steps:

1. Update value function $V(s_t)$, make it close to the estimated cumulative reward (TD target) $r_t + \gamma V(s_{t+1})$. the bracket content in the α is the TD error.

$$V(s_t) \leftarrow V(s_t) + \alpha(r_t + \gamma V(s_{t+1}) - V(s_t))$$

Analysis:

Real target $r_t + \gamma V^\pi(s_{t+1})$ is the unbiased estimation of $V(s_t)$

TD target $r_t + \gamma V(s_{t+1})$ is biased, $\gamma V(s_{t+1})$ is the current estimation

TD target has a lower variance:

Cumulative reward: depend on multi-step random action, multi-step state transition and multi-step reward TD target: depend on single-step random action, single-step state transition and single-step reward

Usually more efficient than MC

TD finally converges to $V^\pi(s_{t+1})$ (but it is not always the case when using function approximation)

More sensitive to initial values than MC

4.3 Multi-step TD Learning

- Instead of updating based on a single reward, we update using a sum of rewards over n steps.
- This helps balance between **speed (shorter horizon)** and **accuracy (longer horizon)**.
- It is useful when time constraints make full-episode learning impractical.
- The update rule still remains model-free because it doesn't rely on knowing the full environment dynamics.

n step cummulative reward:

$$g_t^{(n)} = r_t + \gamma r_{t+1} + \dots + \gamma^{n-1} R_{t+n-1} + \gamma^n R_{t+n}$$

n step TD learning:

$$V(s_t) \leftarrow V(s_t) + \alpha(g_t^{(n)} - V(s_t))$$

4.4 Off-policy MC by importance sampling

Evaluate policy π using the cumulative rewarded generated by policy μ

Cumulative reward g_t should be weighted according by the importance ratio between two policies:

Multiply each segment by the importance ratio:

$$\{s_t, a_t, r_t, s_{t+1}, \dots, s_{T-1}, a_{T-1}, r_{T-1}, s_T, a_T, r_T\} \sim \mu$$

$$g_t^{\pi/\mu} = \frac{\pi(a_t|s_t)}{\mu(a_t|s_t)} \frac{\pi(a_{t+1}|s_{t+1})}{\mu(a_{t+1}|s_{t+1})} \dots \frac{\pi(a_T|s_T)}{\mu(a_T|s_T)} g_t = \prod_{\tau=t}^T \frac{\pi(a_\tau|s_\tau)}{\mu(a_\tau|s_\tau)} g_t$$

Update value function to approximate revised cumulative reward.

Cannot be used when $\pi \neq 0$ but $\mu=0$; and significantly increase variance.

$$V(s_t) \leftarrow V(s_t) + \alpha(g_t^{\pi/\mu} - V(s_t))$$

4.4 Off-policy MC by importance sampling

TD target is weighted by the importance sampling

$$V(s_t) \leftarrow V(s_t) + \alpha \left(\frac{\pi(a_t|s_t)}{\mu(a_t|s_t)} (r_t + \gamma V(s_{t+1})) - V(s_t) \right)$$

Lower variance than MC importance sampling.

The policy only needs to be approximated in a single step.

4.5 ϵ greedy policy improvement

□ Theorem: For any (ϵ -greedy) policy π , ϵ -greedy policy π' based on Q^π is a policy improvement of π

$$V^{\pi'}(s) \geq V^\pi(s)$$

□ Proof:

$$\begin{aligned} V^{\pi'}(s) &\geq Q^\pi(s, \pi'(s)) = \sum_{a \in A} \pi'(a|s) Q^\pi(s, a) \\ &\stackrel{m \text{ actions}}{=} \frac{\epsilon}{m} \sum_{a \in A} Q^\pi(s, a) + (1 - \epsilon) \max_{a \in A} Q^\pi(s, a) \\ &\geq \frac{\epsilon}{m} \sum_{a \in A} Q^\pi(s, a) + (1 - \epsilon) \sum_{a \in A} \frac{\pi(a|s) - \epsilon/m}{1 - \epsilon} Q^\pi(s, a) \\ &= \sum_{a \in A} \pi(a|s) Q^\pi(s, a) = V^\pi(s) \end{aligned}$$

Thinking: for random policies, move the action selection distribution of the policy to actions with higher value.

Note:

1. ϵ cannot be too large
2. π is not ϵ -greedy on Q^π , and π' is

4.6 Summery

Model-free RL

- On-policy MC:

$$V(s_t) \leftarrow V(s_t) + \alpha(g_t - V(s_t))$$

- On-policy TD:

$$V(s_t) \leftarrow V(s_t) + \alpha(r_t + \gamma V(s_{t+1}) - V(s_t))$$

- On-policy TD (SARSA): state-action-reward-state-action

In each time step:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t))$$

Policy improvement: ϵ -greedy

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

Initialize S

Choose A from S using policy derived from Q (e.g., ε -greedy)

Loop for each step of episode:

Take action A , observe R, S'

Choose A' from S' using policy derived from Q (e.g., ε -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

until S is terminal

Note: on-policy TD control use the current policy for action sampling. That is, both " A " in the SARSA algorithm are selected by the current policy.

- Off-policy TD (Q-learning):

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t))$$

Q learning's objective function:

t, get $s_t, a_t, r_t \rightarrow Q. \max_{a'} Q(s_{t+1}, a')$ Select a' , no need to sample.

$$r_{t+1} = \gamma Q(s_{t+1}, a'_{t+1}) = r_{t+1} + \gamma Q(s_{t+1}, \arg \max_{a'} Q(s_{t+1}, a')) = r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a')$$

efficient than SARSA

Do not need importance sampling:

- It **never explicitly computes expectations** over the target policy π , and
- It **doesn't need to reweight samples** using importance sampling because it directly **uses the greedy action** in its update:

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

Initialize S

Loop for each step of episode:

Choose A from S using policy derived from Q (e.g., ε -greedy)

Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

until S is terminal

![image.png](pic/Q-learning convergence.png?t=1743929995986)

5 Multi-step bootstr

折中4, 采样几步就更新几次, 剩下的值函数估计

6 Value-based DRL

这一讲终于跳脱出了前面传统的强化学习（解决相对简单的问题），用神经网络拟合Q, V函数，适用于更大的状态空间。

DQN family

Q-Learning: learns a function $Q_\theta(s, a)$ with para θ

- given a segment $\{(s, a, s', r)\}$
- **target** $y = r + \gamma \max_{a'} Q_\theta(s', a')$
- update:

$$Q_\theta(s, a) \leftarrow Q_\theta(s, a) + \alpha(r + \gamma \max_{a'} Q_\theta(s', a') - Q_\theta(s, a))$$

α 后面类似梯度下降

- optimization objective:

$$\theta^* \leftarrow \arg \min_{\theta} E_{(s, a, s', r) \sim U(D)} \frac{1}{2} [(r + \gamma \max_{a'} Q_\theta(s', a')) - Q_\theta(s, a)]^2$$

- $(r + \gamma \max_{a'} Q_\theta(s', a'))$ TD target, no gradient here???
- $(s, a, s', r) \sim U(D)$: a transition (**state, action, next state, reward**) is **randomly sampled** from the replay buffer D using a **uniform distribution U**.
- $Q_\theta(s', a')$ 如果不固定, 会连续更新, 不稳定

Deep Q-Networks (DQN) represents Q function $Q_\theta(s, a)$ by using neural networks.

- Input:s. Not (s, a) , because too large
- Last layer: a. #of elements $|A|$
- Output:(s,a)

-:

- unstable
 - continuously sampled (s, a, s', r) is not IID
 - $Q_\theta(s', a')$ updates freq
- output is discrete (only fit for discrete action space)

sol:

- Experience replay: non IID data -> IID data
 - Store sample $e_t = (s_t, a_t, s_{t+1}, r_t)$ in each step of training INTO replay buffer D.
 - Sampling, uniformly distributed
- Build 2 nw:
 - Evaluations nw: $Q_\theta(s, a)$
 - Target nw $Q_{\theta^-}(s, a)$ to compute TD target: synchronize with the evaluation nw every C step
 - **target** $y = r + \gamma \max_{a'} Q_{\theta^-}(s', a')$

Algorithm:

1. Randomly initialize evaluation network θ ; $\theta^- \leftarrow \theta$
2. Initialize experience replay buffer D

3. Repeat until convergence{

- Get initial state s_0
- For each step $t=0,1,...,T$
 - take action a_t by ϵ -greedy based on Q_θ
 - Get reward r_t and the next state s_{t+1}
 - Store (s_t, a_t, s_{t+1}, r_t) in D
 - If D is large enough, sampling N samples $\{(s_t, a_t, s_{t+1}, r_t)\}_{i=1}^N$
 - For each sample, calculate target $y_i = r_i + \gamma \max_{a'} Q_{\theta^-}(s_{i+1}, a')$
 - Update θ by minimize loss $L = \frac{1}{2N} \sum_i (y_i - Q_\theta(s_i, a_i))^2$
 - If $t \bmod C=0$, then update $\theta^- \leftarrow \theta$

Double DQN (DDQN), improved version of DQN

Due to the Jensen's inequality, max operation makes the est Q always larger than real Q value. And it will be more serious when #candidate actions increase.

$$\begin{aligned}
 E[\hat{Q}(s, a)] &= Q(s, a) \\
 E[\max_a \hat{Q}(s, a)] &\geq \max_a Q(s, a) \\
 \max_{a'} Q_{\theta^-}(s', a') &= Q_{\theta^-}(s', \arg \max_{a'} Q_{\theta^-}(s', a')) \\
 &= E[R|s', \arg \max_{a'} Q_{\theta^-}(s', a'), \theta^-] \\
 &\geq \max(E[R|s', a_1, \theta^-], E[R|s', a_2, \theta^-], \dots, a_i \in A) \\
 \text{Where } E[\max(X_1, X_2)] &\geq \max(E[X_1], E[X_2])
 \end{aligned}$$

In DDQN, Q_θ is used to select next action.

□ DQN:

$$\begin{aligned}
 y_t &= r_t + \gamma \max_{a'} Q_{\theta^-}(s_{t+1}, a') \\
 &= r_t + \gamma \underbrace{Q_{\theta^-}}_{\text{being a judge in your own case}}(s_{t+1}, \arg \max_{a'} \underbrace{Q_{\theta^-}}_{\text{being a judge in your own case}}(s_{t+1}, a'))
 \end{aligned}$$

□ Double DQN (DDQN) widely used and more accurate

$$y_t = r_t + \gamma \underbrace{Q_{\theta^-}}_{\text{tg nw only to evaluate}}(s_{t+1}, \arg \max_{a'} \underbrace{Q_\theta}_{\text{being a judge in your own case}}(s_{t+1}, a'))$$

Prioritized Experience Replay : find more experience samples

Calculate the priority p_t (value of learning)

$$p_t = |r_t + \gamma Q_{\theta^-}(s_{t+1}, \arg \max_{a'} Q_\theta(s_{t+1}, a')) - Q_\theta(s_t, a_t)|$$

Store experience $e_t = (s_t, a_t, s_{t+1}, r_t, p_t + \epsilon)$, ϵ is the noise for randomness, in replay buffer to give each sample a chance to be sampled.

Prob of e_t being selected is $P(t) = (p_t^\alpha) / (\sum_k p_k^\alpha)$. α for smoothing. ($=0$ in uniform sampling)

Weight in importance sampling $w_t = (N \times P(t))^{-\beta} / (\max_i w_i)$

Algorithm 1 Double DQN with proportional prioritization

```
1: Input: minibatch  $k$ , step-size  $\eta$ , replay period  $K$  and size  $N$ , exponents  $\alpha$  and  $\beta$ , budget  $T$ .
2: Initialize replay memory  $\mathcal{H} = \emptyset$ ,  $\Delta = 0$ ,  $p_1 = 1$ 
3: Observe  $S_0$  and choose  $A_0 \sim \pi_\theta(S_0)$ 
4: for  $t = 1$  to  $T$  do
5:   Observe  $S_t, R_t, \gamma_t$ 
6:   Store transition  $(S_{t-1}, A_{t-1}, R_t, \gamma_t, S_t)$  in  $\mathcal{H}$  with maximal priority  $p_t = \max_{i < t} p_i$ 
7:   if  $t \equiv 0 \pmod K$  then 1st 发现这个sp把优先级放为最大.
8:     for  $j = 1$  to  $k$  do sample k from buf according to pin buffer
9:       Sample transition  $j \sim P(j) = p_j^\alpha / \sum_i p_i^\alpha$ 
10:      Compute importance-sampling weight  $w_j = (N \cdot P(j))^{-\beta} / \max_i w_i$ 
11:      Compute TD-error  $\delta_j = R_j + \gamma_j Q_{\text{target}}(S_j, \arg \max_a Q(S_j, a)) - Q(S_{j-1}, A_{j-1})$ 
12:      Update transition priority  $p_j \leftarrow |\delta_j|$ 
13:      Accumulate weight-change  $\Delta \leftarrow \Delta + w_j \cdot \delta_j \cdot \nabla_\theta Q(S_{j-1}, A_{j-1})$ 
14:    end for
15:    Update weights  $\theta \leftarrow \theta + \eta \cdot \Delta$ , reset  $\Delta = 0$ 
16:    From time to time copy weights into target network  $\theta_{\text{target}} \leftarrow \theta$ 
17:  end if
18:  Choose action  $A_t \sim \pi_\theta(S_t)$ 
19: end for
```

Dueling DQN, improved version of DQN. parallel with DDQN

Inputs of s are processed by CNN to extract features.

Core: Advantage function A

$$\begin{aligned} A^\pi(s, a) &= Q^\pi(s, a) - V^\pi(s) \\ Q^\pi(s, a) &= E[G_t | s, a] \\ V^\pi(s) &= E_{a \sim \pi(\cdot|s)}[Q^\pi(s, a)] \end{aligned}$$

$V^\pi(s)$ is the average value of $Q^\pi(s, a)$. $A^\pi(s, a)$ measures the effect after taking an action.

To obtain V, A separately (fc layer), Q in the DQN is decomposed. And they are combined into Q finally. *Subscript denotes parameters.*

$$Q_{\theta, \alpha, \beta}(s, a) = V_{\theta, \alpha}(s) + A_{\theta, \beta}(s, a)$$

However, this version is unstable in training because the non-uniqueness in modeling V and A .

For sol 1, max two sides of $Q=V+A$ (max limits a only operates the f with para a), define the optimal A function $(\max_a A(s, a) - \max_a A(s, a))$ equals to 0.

$$\begin{aligned} Q(s, a) &= V(s) + A(s, a) \\ \max_a Q(s, a) &= V(s) + \max_a A(s, a) - \max_a A(s, a) \\ V(s) &= \max_a Q(s, a) \end{aligned}$$

So is the sol 2.

Therefore the two sols:

$$1. \text{Set } V_{\theta, \alpha}(s) = \max_{a'} Q_{\theta, \alpha, \beta}(s, a')$$

$$Q_{\theta, \alpha, \beta}(s, a) = V_{\theta, \alpha}(s) + A_{\theta, \beta}(s, a) - \max_{a'} A_{\theta, \beta}(s, a')$$

$$2. \text{Set } V_{\theta, \alpha}(s) = \frac{1}{|A|} \sum_{a' \in A} Q_{\theta, \alpha, \beta}(s, a')$$

$$Q_{\theta, \alpha, \beta}(s, a) = V_{\theta, \alpha}(s) + A_{\theta, \beta}(s, a) - \frac{1}{|A|} \sum_{a' \in A} A_{\theta, \beta}(s, a')$$

The sol ensure the uniqueness of V function,

but not satisfy with the Bellman f, the outputs of A, V, Q of the network are no longer the real A, V, Q .

We do not care of it, because the standard of doing greedy is the order of Q .

The relative order of Q remains the same. s.t. $Q(s, a_1) > Q(s, a_2) \rightarrow A(s, a_1) > A(s, a_2)$

- +:
 - Handle states that are less associated with actions. 没人的路上怎么开都行。
 - effective in learning state-value f: one state value function corresponds to multiple Advantage. functions. Share the same state-value function; Easy Training, fast convergence.

7 Stochastic Policy Gradient (SPG)

This chapter considers the neural network that model the policy directly.

1. Parameterized Policy

Model the policy

- stochastic $\pi_\theta(a|s) = P(a|s; \theta)$ output: a prob
- deterministic $a = \pi_\theta(s)$ output: an action
 θ : nn's para to model the policy
 +: Generalize the visible known state to the unknown state.(For value based method, to get Q(s,a), must get (s,a) pair fist)

2. Policy-based RL

- +:
 - better convergence, effective in high-dimensional/continuous action space.
Value based method: $a^* = \max_a Q(s, a)$, comparison of a list by list, and discrete A.
 - can learn stochastic policy
- -:
 - often converge to the local optimum, not global optimum. (nn is not convex)
 - inefficient in evaluating a policy & large variance.
- objective
 - policy learning $J(\theta) = E_{\tau \sim \pi_\theta(\tau)}[G(\tau)] = E_{s_0 \sim v_0}[V^{\pi^\theta}(s_0)]$
 - $G(\tau)$ discounted cummulative return of trajectory τ
 - v_0 distribution of initial states.
 - more forms

$$\begin{aligned}
 J(\theta) &= E_{\tau \sim \pi_\theta(\tau)}[G(\tau)] \\
 &= E_{\tau \sim \pi_\theta(\tau)}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)\right] \\
 &= E_{\pi_\theta}\left[\sum_{t=0}^{\infty} \gamma^t r(S_t, A_t)\right] \\
 &= E_{(s,a) \sim \tilde{\rho}^{\pi_\theta}}[r(s, a)] \\
 &= \frac{1}{1-\gamma} E_{(s,a) \sim \rho^{\pi_\theta}}[r(s, a)]
 \end{aligned}$$

- Policy gradient in 1-step MDP

Starting state $s \sim d(s)$, The MDP ends after one-step decision-making, and reward is $r(s, a)$

dft

Approximate Value Functions

- $v_\theta(s)$: approximate value of state s given parameter vector θ
- $q_\theta(s, a)$: approximate value of state-action pair (s, a) given parameter vector θ
- $\nabla v_\theta(s)$: column vector of partial derivatives of $v_\theta(s)$ with respect to θ
- $\nabla q_\theta(s, a)$: column vector of partial derivatives of $q_\theta(s, a)$ with respect to θ

Bellman Operators

- B_π : Bellman operator for value functions
- P : state-transition probability matrix under π
- D : diagonal matrix with on-policy state distribution on its diagonal
- X : feature matrix, with $x(s)$ as its rows
- Π : projection operator for value functions

Missing Symbols and Notation

Below are the missing symbols and notation that were not properly formatted in the original document:

Symbols

- $\mathbb{R}$: set of real numbers
- $f : X \rightarrow Y$: function f from elements of set X to elements of set Y
- $(a, b]$: the real interval between a and b including b but not including a
- $\in$: is an element of; e.g., $s \in S, r \in R$
- $\subseteq$: subset of; e.g., $R \subseteq \mathbb{R}$
- $|S|$: number of elements in set S

Temporal Difference Learning

- δ_t : temporal-difference (TD) error at time t
- δ_t^s, δ_t^a : state- and action-specific forms of the TD error
- n : in n -step methods, n is the number of steps of bootstrapping

Eligibility Traces

- w, w_t : weight vector in function approximation
- $\mathbf{w}, \mathbf{w}_t$: weight vector notation
- $x(s)$: vector of features visible in state s
- $x(s, a)$: vector of features visible in state s taking action a
- $\mathbf{x}(s), \mathbf{x}(s, a)$: feature vectors in bold
- $\mathbf{w}^T \mathbf{x}$: inner product of weight vector and feature vector

Policy Gradient Methods

- θ, θ_t : parameter vector of target policy
- $\pi(a|s, \theta)$: probability of taking action a in state s given parameter θ
- $\nabla \pi(a|s, \theta)$: gradient of policy function
- $J(\theta)$: performance measure for policy π_θ
- $\nabla J(\theta)$: gradient of performance measure
- $b(a|s)$: behavior policy used to select actions while learning about target policy π

Importance Sampling

- $\rho_t : h$: importance sampling ratio for time t through time h
- ρ_t : importance sampling ratio for time t alone, $\rho_t = \rho_{t:t}$
- $r(\pi)$: average reward (reward rate) for policy π
- $\bar{R}_t$: estimate of $r(\pi)$ at time t

Norms and Errors

- $\|v\|_\mu^2$: μ -weighted squared norm of value function v
- $\|\delta_t\|^2$: squared temporal-difference error
- $BE(w)$: mean square Bellman error
- $PBE(w)$: mean square projected Bellman error
- $TDE(w)$: mean square temporal-difference error

References

1. Sutton, R.,S. & Barto, A.,G. (2018). Reinforcement learning: An introduction. (2 e.d.)
2. Lecture notes by Jianxiong Guo.